Informe — Guía 3: Raíces de ecuaciones por métodos numéricos

Alumnos: Franco Zambrano · Gabriel Bazzola

Métodos Computacionales · Laboratorio 3 · Octave 6.4.0

Ejercicio 1 — Tanque esférico

`01-tanque-esferico.m`

```
% 01-tanque-esferico.m - Ejercicio 1 de la Guía 3: raíces de una ecuación.
%
% Un tanque esférico de radio R = 3 m debe contener V = 30 m^3. El volumen de
% agua a profundidad h es
%   V(h) = pi * h^2 * (3R - h)/3,
% así que la profundidad buscada es la raíz de f(h) = V(h) - V = 0.
%
% Se grafica f con fplot, se separan las raíces con Tanteo y se aproximan con
% Intervalo Medio e Interpolación Lineal hasta E < 0.001.

clc;

% ---------------------------------------------------------------------
% Datos del problema
% ---------------------------------------------------------------------
R = 3;   % radio del tanque en metros
V = 30;  % volumen objetivo en metros cúbicos

% Los operadores .* y .^ son obligatorios: fplot evalúa f sobre vectores.
f = @(h) pi * (h.^2) .* ((3 * R - h) / 3) - V;

% Rango de búsqueda: h va de 0 (vacío) a 2R = 6 (lleno hasta el ecuador).
% Más allá del ecuador el polinomio sigue definido, pero ya no da volúmenes.
h_min = 0;
h_max = 2 * R;

fprintf('\n=========================================================\n');
fprintf('  EJERCICIO 1 - TANQUE ESFÉRICO\n');
fprintf('  V = pi*h^2*(3R - h)/3   con   R = %.1f m,  V = %.1f m^3\n', R, V);
fprintf('=========================================================\n');

% ---------------------------------------------------------------------
% a) Evaluación gráfica de las raíces con fplot
% ---------------------------------------------------------------------
figure('Name', 'Ejercicio 1 - Tanque esférico');
fplot(f, [h_min, h_max], 'b-', 'LineWidth', 2);
grid on;
hold on;

% Eje horizontal en y = 0: los cruces con esta recta son las raíces.
plot([h_min, h_max], [0, 0], 'r--', 'LineWidth', 1.5);

title('Evaluación gráfica: f(h) = V(h) - 30');
xlabel('Profundidad h (m)');
ylabel('f(h)');
legend({'f(h)', 'y = 0'}, 'Location', 'northeast');

fprintf('\n--- a) Evaluación gráfica ---\n');
fprintf('f(h) vale %.4f en h = %.1f y %.4f en h = %.1f.\n', ...
```

```
        f(h_min), h_min, f(h_max), h_max);

% ------------------------------------------------------------------------
% b) Separación de las raíces con el Método de Tanteo
% ------------------------------------------------------------------------
fprintf('\n--- b) Separación de raíces con Tanteo ---\n');
intervalos = tanteo(f, h_min, h_max);

if (isempty(intervalos))
  fprintf('\nNo se separó ninguna raíz: revise el incremento delta_x.\n');
  return;
endif

fprintf('\nSe separaron %d raíz/raíces.\n', rows(intervalos));

% ------------------------------------------------------------------------
% c) Aproximación con Intervalo Medio e Interpolación Lineal
% ------------------------------------------------------------------------
tol = input(sprintf('\nIngrese la cota de error a usar (se sugiere 0.001): '));

for k = 1:rows(intervalos)
  a = intervalos(k, 1);
  b = intervalos(k, 2);

  fprintf('\n========== Intervalo %d: [%.4f, %.4f] ==========\n', k, a, b);

  % --- Método de Intervalo Medio (bisección) ---
  tic;
  [raiz_im, iter_im] = intervalo_medio(f, a, b, tol);
  tiempo_im = toc;

  fprintf('\nMétodo de Intervalo Medio:\n');
  fprintf('  raíz = %.6f m | f(raíz) = %.3e | iteraciones = %d | tiempo = %.6f s\n', ...
          raiz_im, f(raiz_im), iter_im, tiempo_im);

  % --- Método de Interpolación Lineal ---
  tic;
  [raiz_il, iter_il, tabla_il] = interpolacion_lineal(f, a, b, tol);
  tiempo_il = toc;

  fprintf('\nMétodo de Interpolación Lineal:\n');
  fprintf('  %-6s %-12s %-12s %-12s\n', 'Iter', 'a', 'b', 'c');
  for i = 1:rows(tabla_il)
    fprintf('  %-6d %-12.6f %-12.6f %-12.6f\n', tabla_il(i, 1), tabla_il(i, 2), ...
            tabla_il(i, 3), tabla_il(i, 4));
  endfor
  fprintf('  raíz = %.6f m | f(raíz) = %.3e | iteraciones = %d | tiempo = %.6f s\n', ...
          raiz_il, f(raiz_il), iter_il, tiempo_il);

  % Se marcan ambas raíces sobre el gráfico del punto a.
  plot(raiz_im, f(raiz_im), 'go', 'MarkerSize', 8, 'LineWidth', 2);
  plot(raiz_il, f(raiz_il), 'ms', 'MarkerSize', 8, 'LineWidth', 2);
endfor

hold off;
```

Salida de ejecución

```
Ejercicio 1 - Tanque esferico
============================================================
  EJERCICIO 1 - TANQUE ESFÉRICO
  V = pi*h^2*(3R - h)/3   con   R = 3.0 m,  V = 30.0 m^3
============================================================

--- a) Evaluación gráfica ---
f(h) vale -30.0000 en h = 0.0 y 83.0973 en h = 6.0.

--- b) Separación de raíces con Tanteo ---
--- METODO DE TANTEO ---
Ingrese el valor del incremento delta_x (ej. 0.5 o 0.1):
--- Tabla de Valores del Método de Tanteo ---
x          f(x)
0.0000     -30.0000
0.5000     -27.7747
1.0000     -21.6224
1.5000     -12.3285
2.0000     -0.6785
2.5000     12.5424
>>> Se detectó cambio de signo en el intervalo: [2.0000, 2.5000]
3.0000     26.5487
3.5000     40.5549
4.0000     53.7758
4.5000     65.4259
5.0000     74.7198
5.5000     80.8720
6.0000     83.0973

Raíz separada con éxito para el paso siguiente en el intervalo:
[2, 2.5]

Se separaron 1 raíz/raíces.

Ingrese la cota de error a usar (se sugiere 0.001):
=========== Intervalo 1: [2.0000, 2.5000] ===========

Método de Intervalo Medio:
  raíz = 2.026367 m | f(raíz) = -1.362e-02 | iteraciones = 8 | tiempo = 0.000382 s

Método de Interpolación Lineal:
  Iter   a            b            c
  1      2.000000     2.500000     2.025659
  2      2.025659     2.500000     2.026849
  3      2.026849     2.500000     2.026903
  raíz = 2.026903 m | f(raíz) = -6.595e-05 | iteraciones = 3 | tiempo = 0.000448 s
```

Salida del ejercicio 1

Gráfico

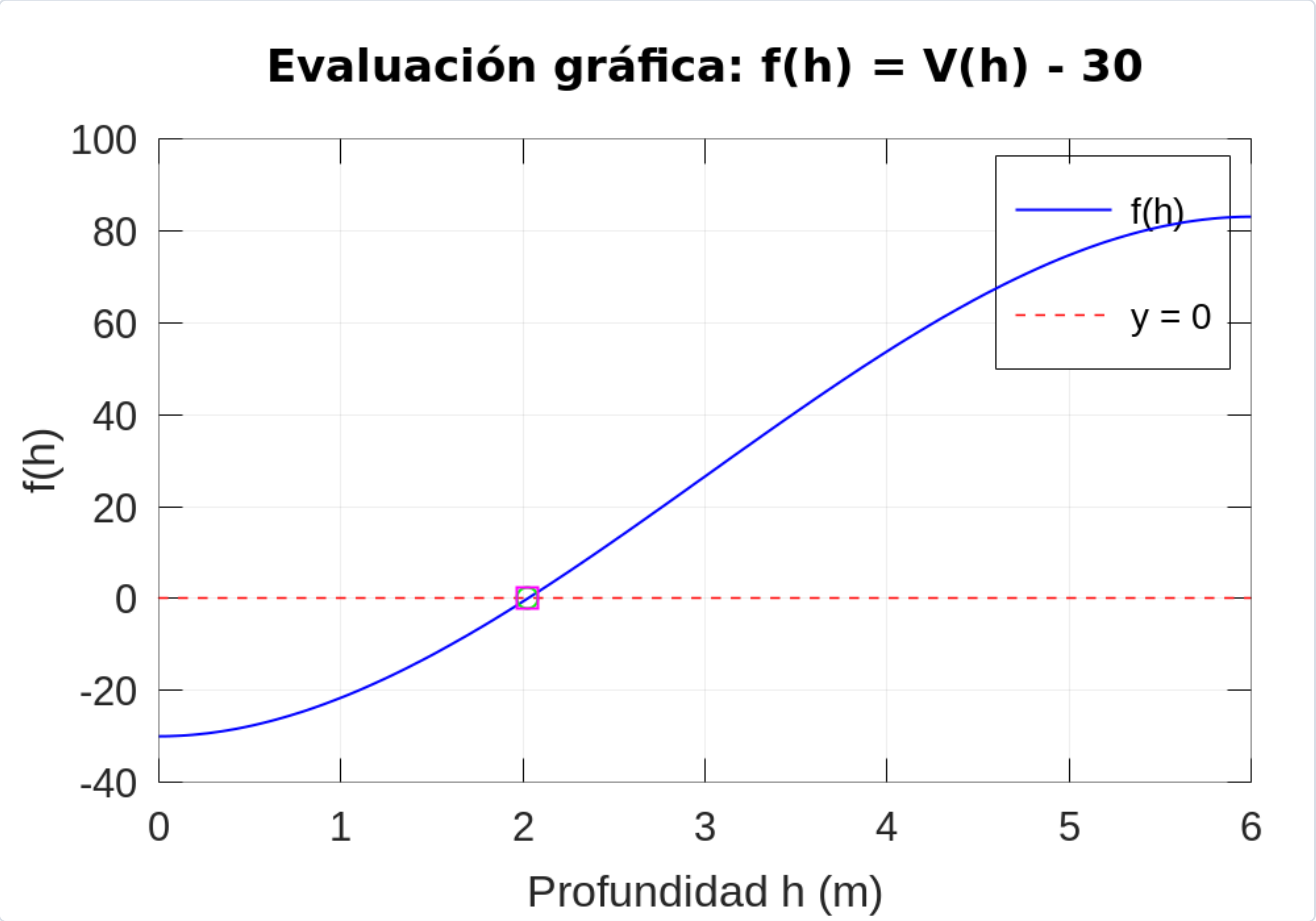


Gráfico del ejercicio 1

Ejercicio 2 — Punto de equilibrio de impresoras

`02-punto-equilibrio-impresoras.m`

```
% 02-punto-equilibrio-impresoras.m - Ejercicio 2 de la Guía 3: Newton-Raphson.
%
% Utilidad total de un fabricante de impresoras, en pesos:
%   P(x) = 8x + 0.3x^2 - 0.0013x^3 - 372
% Los puntos de equilibrio son las raíces de P(x) = 0. Se grafican con fplot,
% se separan con Tanteo y se aproximan con Newton-Raphson hasta E < 1e-3,
% eligiendo x0 según las condiciones de convergencia.

clc;

% ---------------------------------------------------------------------
% Datos del problema
% ---------------------------------------------------------------------
% Operadores element-wise (.*, .^) obligatorios: fplot evalúa vectores.
F = @(x) 8 * x + 0.3 * (x.^2) - 0.0013 * (x.^3) - 372;

% Rango de graficado: alcanza para ver las dos raíces y el máximo.
x_min = 0;
x_max = 300;

% Paso para aproximar derivadas por diferencias finitas.
h = 1e-4;
dF  = @(x) (F(x + h) - F(x - h)) / (2 * h);                % F'(x)  por diferencia central
d2F = @(x) (F(x + h) - 2 * F(x) + F(x - h)) / (h ^ 2);     % F''(x) por diferencias centradas

fprintf('\n============================================================\n');
fprintf('  EJERCICIO 2 - PUNTO DE EQUILIBRIO DE IMPRESORAS\n');
fprintf('  P(x) = 8x + 0.3x^2 - 0.0013x^3 - 372\n');
fprintf('============================================================\n');

% ---------------------------------------------------------------------
% a) Raíces en forma gráfica
% ---------------------------------------------------------------------
figure('Name', 'Ejercicio 2 - Punto de equilibrio');
fplot(F, [x_min, x_max], 'b-', 'LineWidth', 2);
grid on;
hold on;

% Eje horizontal en y = 0: los cruces son los puntos de equilibrio.
plot([x_min, x_max], [0, 0], 'r--', 'LineWidth', 1.5);

title('Punto de equilibrio: F(x) = P(x) = 0');
xlabel('x (impresoras por mes)');
ylabel('P(x) (pesos)');
legend({'P(x)', 'y = 0'}, 'Location', 'northeast');

fprintf('\n--- a) Evaluación gráfica ---\n');
fprintf('P(0) = %.2f pesos.\n', F(0));
fprintf('P(300) = %.2f pesos.\n', F(300));

% Separación de las raíces con Tanteo sobre el rango graficado.
fprintf('\n--- Separación de raíces con Tanteo ---\n');
intervalos = tanteo(F, x_min, x_max);

if (isempty(intervalos))
  fprintf('\nNo se separó ninguna raíz: revise el incremento delta_x.\n');
  return;
endif

fprintf('\nSe separaron %d punto(s) de equilibrio.\n', rows(intervalos));
fprintf('Compare con los intervalos del problema: [24, 26] y [250, 252].\n');
```

```
% ----------------------------------------------------------------------
% b) Newton-Raphson con análisis previo de las condiciones de Fourier
% ----------------------------------------------------------------------
tol = input(sprintf('\nIngrese la cota de error a usar (se sugiere 0.001): '));

for k = 1:rows(intervalos)  a = intervalos(k, 1);
  b = intervalos(k, 2);

  fprintf('\n============ Intervalo %d: [%.2f, %.2f] ============\n', k, a, b);

  % --- Condiciones de Fourier ---
  % Se necesitan las tres para que Newton-Raphson converja hacia la raíz de [a, b]:
  %   1. F(a)*F(b) < 0            (Bolzano: hay una raíz adentro).
  %   2. F'(x) != 0 en [a, b]      (se puede dividir por la derivada).
  %   3. F''(x) no cambia de signo (la raíz es simple, la tangente no oscila).
  fprintf('\nAnálisis de las condiciones de Fourier:\n');

  fa = F(a);
  fb = F(b);
  fprintf('  1. F(a)*F(b) = %.4f -> %s\n', fa * fb, ...
          merge(fa * fb < 0, 'cambio de signo: la raíz está en el intervalo', ...
                'SIN cambio de signo: no hay raíz garantizada'));

  % F' muestreada sobre el intervalo para ver si se anula.
  malla = linspace(a, b, 100);
  df_malla = dF(malla);
  derivada_nula = any(abs(df_malla) < 1e-8);
  fprintf('  2. F''(x) en [%.2f, %.2f]: min = %.4f, max = %.4f -> %s\n', ...
          a, b, min(df_malla), max(df_malla), ...
          merge(derivada_nula, 'F'' SE ANULA: no se puede dividir', ...
                'F'' no se anula: se puede dividir'));

  % F'' muestreada para ver si cambia de signo en [a, b].
  d2f_malla = d2F(malla);
  d2f_cambio_signo = (min(d2f_malla) < 0 && max(d2f_malla) > 0);
  fprintf('  3. F''''(x) en [%.2f, %.2f]: min = %.4f, max = %.4f -> %s\n', ...
          a, b, min(d2f_malla), max(d2f_malla), ...
          merge(d2f_cambio_signo, 'CAMBIA de signo: no se cumple', ...
                'no cambia de signo: se cumple'));

  % --- Elección de x0 ---
  % Se arranca en el extremo donde F(x0)*F''(x0) > 0: ahí la tangente apunta
  % hacia la raíz y la iteración no se escapa del intervalo.
  if (fa * d2F(a) > 0)
    x0 = a;
  else
    x0 = b;
  endif
  fprintf('\nF(x0)*F''''(x0) > 0 en x0 = %.4f (F = %.4f, F'''' = %.4f) -> se arranca en %s\n\n', ...
          x0, F(x0), d2F(x0), merge(x0 == a, 'a', 'b'));

  % --- Newton-Raphson ---
  tic;
  [raiz, iter, convergio] = newton_raphson(F, x0, tol);
  tiempo = toc;

  if (!convergio)
    fprintf('Newton-Raphson no convergió desde x0 = %.4f.\n', x0);
    continue;
  endif

  fprintf('>>> Punto de equilibrio: x = %.4f impresoras (aprox. %d).\n', raiz, ceil(raiz));
  fprintf('>>> Iteraciones: %d | Tiempo: %.6f s | F(raíz) = %.3e\n', iter, tiempo, F(raiz));

  % Se marca la raíz sobre el gráfico del punto a.
  plot(raiz, 0, 'ko', 'MarkerSize', 8, 'LineWidth', 2);
  text(raiz, 0, sprintf('  x_%.0f = %.2f', k, raiz), ...
```

```
       'VerticalAlignment', 'bottom', 'FontSize', 9);
endfor

hold off;
```

Salida de ejecución

```
Ejercicio 2 - Punto de equilibrio
=========================================================
  EJERCICIO 2 - PUNTO DE EQUILIBRIO DE IMPRESORAS
  P(x) = 8x + 0.3x^2 - 0.0013x^3 - 372
=========================================================

--- a) Evaluación gráfica ---
P(0) = -372.00 pesos.
P(300) = -6072.00 pesos.

--- Separación de raíces con Tanteo ---
--- METODO DE TANTEO ---
Ingrese el valor del incremento delta_x (ej. 0.5 o 0.1):
--- Tabla de Valores del Método de Tanteo ---
x          f(x)
0.0000     -372.0000
20.0000    -102.4000
40.0000    344.8000
>>> Se detectó cambio de signo en el intervalo: [20.0000, 40.0000]
60.0000    907.2000
80.0000    1522.4000
100.0000   2128.0000
120.0000   2661.6000
140.0000   3060.8000
160.0000   3263.2000
180.0000   3206.4000
200.0000   2828.0000
220.0000   2065.6000
240.0000   856.8000
260.0000   -860.8000
>>> Se detectó cambio de signo en el intervalo: [240.0000, 260.0000]
280.0000   -3149.6000
300.0000   -6072.0000

Raíz separada con éxito para el paso siguiente en el intervalo:
[20, 40]
[240, 260]

Se separaron 2 punto(s) de equilibrio.
Compare con los intervalos del problema: [24, 26] y [250, 252].

Ingrese la cota de error a usar (se sugiere 0.001):
============ Intervalo 1: [20.00, 40.00] ============

Análisis de las condiciones de Fourier:
  1. F(a)*F(b) = -35307.5200 -> cambio de signo: la raíz está en el intervalo
  2. F'(x) en [20.00, 40.00]: min = 18.4400, max = 25.7600 -> F' no se anula: se puede dividir
  3. F''(x) en [20.00, 40.00]: min = 0.2880, max = 0.4440 -> no cambia de signo: se cumple

F(x0)*F''(x0) > 0 en x0 = 40.0000 (F = 344.8000, F'' = 0.2880) -> se arranca en b

>>> Punto de equilibrio: x = 25.2335 impresoras (aprox. 26).
>>> Iteraciones: 4 | Tiempo: 0.000621 s | F(raíz) = 1.876e-12

============ Intervalo 2: [240.00, 260.00] ============

Análisis de las condiciones de Fourier:
  1. F(a)*F(b) = -737533.4400 -> cambio de signo: la raíz está en el intervalo
  2. F'(x) en [240.00, 260.00]: min = -99.6400, max = -72.6400 -> F' no se anula: se puede dividir
  3. F''(x) en [240.00, 260.00]: min = -1.4283, max = -1.2718 -> no cambia de signo: se cumple

F(x0)*F''(x0) > 0 en x0 = 260.0000 (F = -860.8000, F'' = -1.4275) -> se arranca en b

>>> Punto de equilibrio: x = 250.7593 impresoras (aprox. 251).
>>> Iteraciones: 4 | Tiempo: 0.000191 s | F(raíz) = 0.000e+00
```

Salida del ejercicio 2

Gráfico

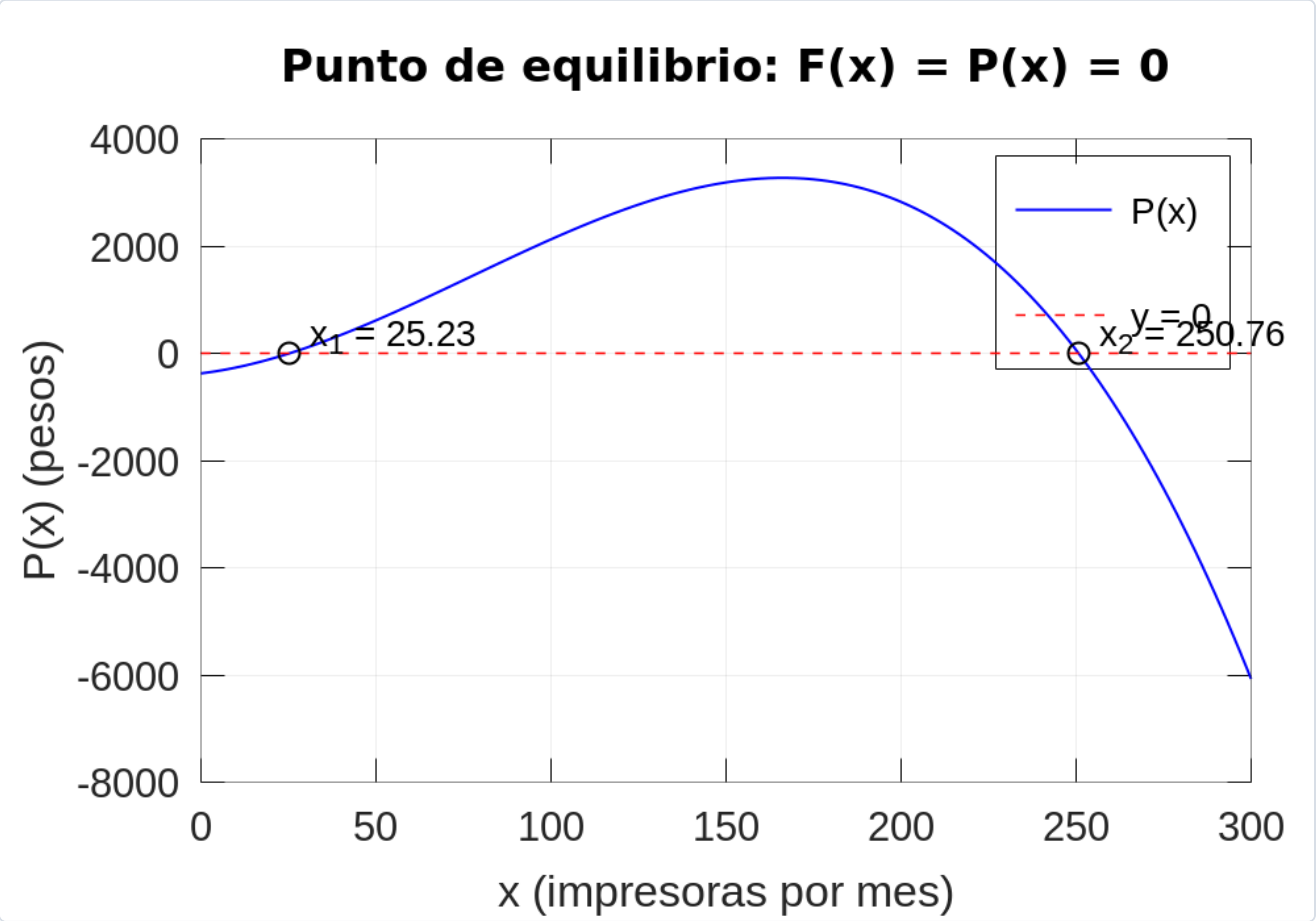


Gráfico del ejercicio 2

Ejercicio 3 — Publicidad y utilidad neta

`03-publicidad-utilidad.m`

```
% 03-publicidad-utilidad.m - Ejercicio 3 de la Guía 3: iteración y Aitken.
%
% Una compañía gasta A(x) = 200*ln(400/(500 - x)) pesos por semana en publicidad
% para vender x unidades a $5. La utilidad neta es R(x) = 5x - A, y se busca la
% x que da R(x) = 1000.
%
% Despejando x se llega a la ecuación de punto fijo
%   g(x) = 200 + 40*ln(400/(500 - x)),  con x0 = 200,
% que se resuelve por iteración simple y por Aitken, ambas hasta E < 1e-4.

clc;

% ---------------------------------------------------------------------
% Datos del problema
% ---------------------------------------------------------------------
% Operadores element-wise (./) obligatorios: fplot evalúa vectores.
F = @(x) 5 * x - 200 * log(400 ./ (500 - x)) - 1000;  % raíz a buscar
g = @(x) 200 + 40 * log(400 ./ (500 - x));            % función de iteración

fprintf('\n============================================================\n');
fprintf('  EJERCICIO 3 - PUBLICIDAD Y UTILIDAD NETA\n');
fprintf('  A(x) = 200*ln(400/(500 - x)),  R(x) = 5x - A\n');
fprintf('  Buscar R(x) = 1000\n');
fprintf('============================================================\n');

% Paso para aproximar g' por diferencia central.
h = 1e-4;
dg = @(x) (g(x + h) - g(x - h)) / (2 * h);

fprintf('\n');
x0      = input('Ingrese el valor inicial x0 (el problema propone 200): ');
tol     = input('Ingrese la cota de error a usar (se sugiere 0.0001): ');

% Condición de convergencia del punto fijo: |g'(x0)| < 1. Acá g'(x) = 40/(500 - x)
% vale 0.133 en x0 = 200, pero crece sin límite cuando x se acerca a 500.
g_primo_x0 = dg(x0);
fprintf("\n|g'(x0)| = |%.6f| = %.6f -> %s\n", g_primo_x0, abs(g_primo_x0), ...
        merge(abs(g_primo_x0) < 1, ...
              'punto fijo CONVERGE desde este x0', ...
              'punto fijo DIVERGE desde este x0 (|g''(x0)| >= 1)'));

% ---------------------------------------------------------------------
% Método de iteración simple
% ---------------------------------------------------------------------
fprintf('\n--- a) Método de Iteración (punto fijo simple) ---\n');
tic;
[x_simple, iter_simple, ok_simple] = iteracion(g, x0, tol);
tiempo_simple = toc;

if (ok_simple)
  fprintf('Raíz aproximada: x = %.6f unidades (aprox. %d)\n', x_simple, ceil(x_simple));
  fprintf('Iteraciones: %d | Tiempo: %.6f s | F(raíz) = %.3e\n', ...
          iter_simple, tiempo_simple, F(x_simple));
else
  fprintf('El punto fijo no convergió desde x0 = %.4f: |g''(x0)| >= 1.\n', x0);
  x_simple = NaN;
  iter_simple = NaN;
  tiempo_simple = NaN;
endif

% ---------------------------------------------------------------------
```

```
% Método de iteración acelerado con Aitken
% -------------------------------------------------------------------------
fprintf('\n--- b) Método de Iteración con Aceleración de Aitken ---\n');

tic;
[x_aitken, iter_aitken, ok_aitken] = aceleracion(g, x0, tol);
tiempo_aitken = toc;

if (ok_aitken)
  fprintf('Raíz aproximada: x = %.6f unidades (aprox. %d)\n', x_aitken, ceil(x_aitken));
  fprintf('Iteraciones: %d | Tiempo: %.6f s | F(raíz) = %.3e\n', ...
          iter_aitken, tiempo_aitken, F(x_aitken));
else
  fprintf('Aitken no pudo acelerar: se anuló el denominador.\n');
  x_aitken = NaN;
  iter_aitken = NaN;
  tiempo_aitken = NaN;
endif

% -------------------------------------------------------------------------
% Gráfico: y = x contra y = g(x), las dos curvas en la misma figura
% -------------------------------------------------------------------------
figure('Name', 'Ejercicio 3 - Punto fijo');

% Rango centrado en x0, acotado para que se vea bien el punto fijo.
rango = [max(1, x0 - 60), x0 + 60];

% Recta y = x: sus cruces con g son los puntos fijos.
fplot(@(x) x, rango, 'r--', 'LineWidth', 1.5);
hold on;
grid on;

fplot(g, rango, 'b-', 'LineWidth', 2);

if (ok_simple)
  plot(x_simple, x_simple, 'ko', 'MarkerSize', 8, 'LineWidth', 2);
endif

title('Punto Fijo: Intersección entre y = x  e  y = g(x)');
xlabel('x (unidades semanales)');
ylabel('y');
leyenda = {'y = x', 'y = g(x)'};
if (ok_simple)
  leyenda = [leyenda, {sprintf('Punto fijo (%.2f)', x_simple)}];
endif
legend(leyenda, 'Location', 'NorthWest');
hold off;
```

Salida de ejecución

```
Ejercicio 3 - Publicidad y utilidad
======================================================
  EJERCICIO 3 - PUBLICIDAD Y UTILIDAD NETA
  A(x) = 200*ln(400/(500 - x)),  R(x) = 5x - A
  Buscar R(x) = 1000
======================================================

Ingrese el valor inicial x0 (el problema propone 200): Ingrese la cota de error a usar (se sugiere 0.0001):
|g'(x0)| = |0.133333| = 0.133333 -> punto fijo CONVERGE desde este x0

--- a) Método de Iteración (punto fijo simple) ---
Raíz aproximada: x = 213.324539 unidades (aprox. 214)
Iteraciones: 7 | Tiempo: 0.000519 s | F(raíz) = -5.748e-05

--- b) Método de Iteración con Aceleración de Aitken ---
Raíz aproximada: x = 213.324553 unidades (aprox. 214)
Iteraciones: 3 | Tiempo: 0.000521 s | F(raíz) = -1.137e-13
```

Salida del ejercicio 3

Gráfico

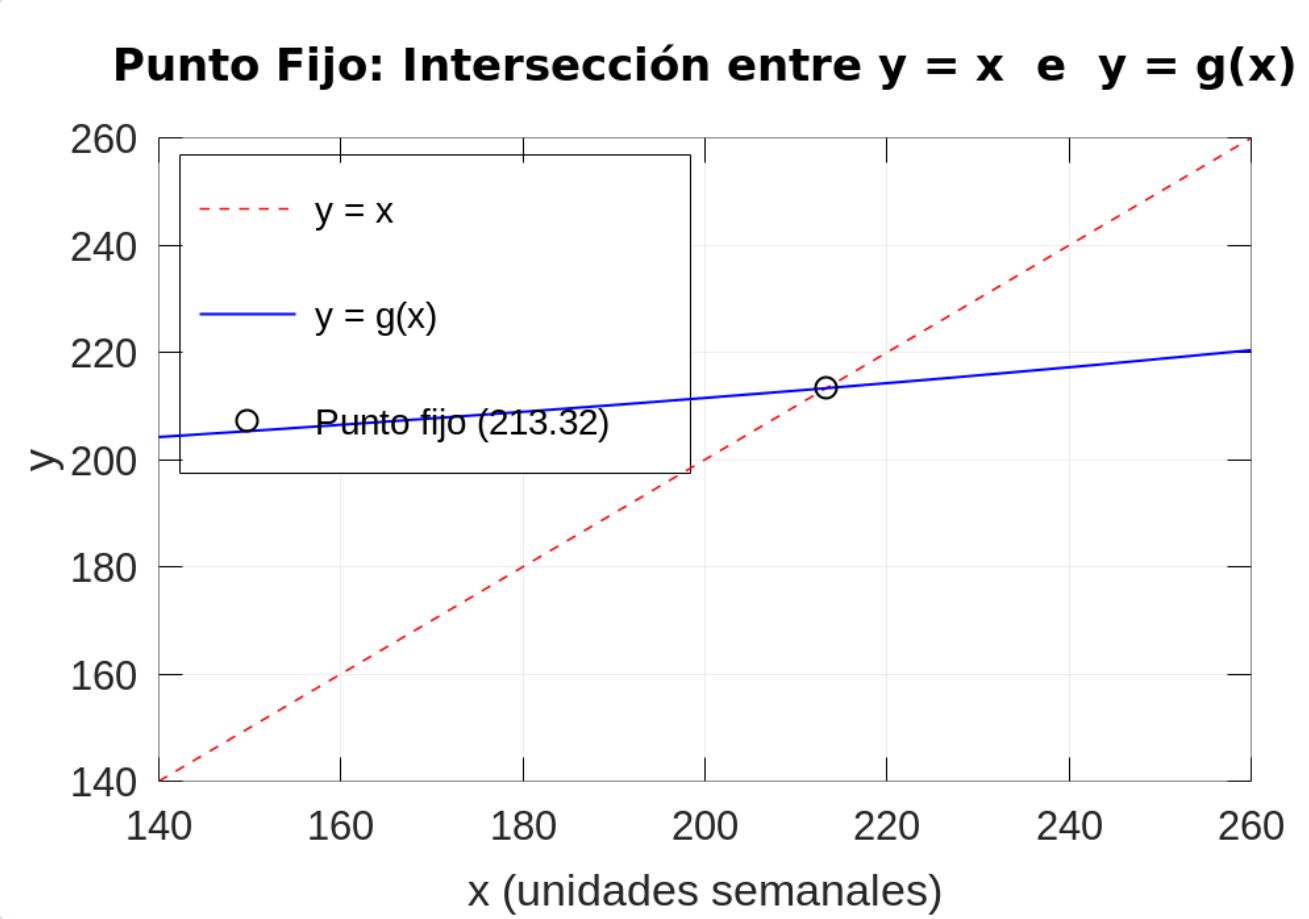


Gráfico del ejercicio 3

Anexo — Métodos numéricos

Tanteo

```
function intervalos = tanteo(f, a, b)
  % TANTEO - Separa raíces barriendo [a, b] con paso delta_x y detectando cambios
  % de signo.
  % Entradas:
  %   f - función a la que se le buscan raíces, ej: @(x) x.^2 - 2
  %   a - extremo inferior del barrido
  %   b - extremo superior del barrido
  % Salida:
  %   intervalos - matriz de Nx2 con los [a, b] donde f cambia de signo (vacía si no hay)
  if (nargin != 3)
    error("tanteo: se requieren 3 argumentos (f, a, b)");
  endif
  if (!isa(f, "function_handle"))
    error("tanteo: f debe ser un function_handle, ej: @(x) x.^2 - 2");
  endif

  disp('--- METODO DE TANTEO ---');

  % El paso debe ser un número positivo menor que el ancho del intervalo.
  while (true)
    delta_x = str2double(input('Ingrese el valor del incremento delta_x (ej. 0.5 o 0.1): ', 's'));
    if (isscalar(delta_x) && isfinite(delta_x) && delta_x > 0 && delta_x <= (b - a))
      break;
    endif
    disp('Paso invalido: debe ser un numero positivo no mayor que (b - a)');
  endwhile

  % Punto de partida: extremo inferior del rango.
  x_act = a;
  f_act = f(x_act);

  % Tabla de valores del barrido: x contra f(x).
  fprintf('\n--- Tabla de Valores del Método de Tanteo ---\n');
  fprintf('%-10s %-15s\n', 'x', 'f(x)');
  fprintf('%-10.4f %-15.4f\n', x_act, f_act);

  % Cada fila acumulada es un [x_act, x_sig] con cambio de signo.
  intervalos_encontrados = [];

  % Barrido de [a, b] con paso delta_x.
  while (x_act + delta_x) <= b
    x_sig = x_act + delta_x; % siguiente punto del barrido
    f_sig = f(x_sig);
    fprintf('%-10.4f %-15.4f\n', x_sig, f_sig);

    % Si f cambia de signo (o toca cero) hay una raíz entre x_act y x_sig.
    if (f_act * f_sig) <= 0
      fprintf('>>> Se detectó cambio de signo en el intervalo: [%.4f, %.4f]\n', x_act, x_sig);
      intervalos_encontrados = [intervalos_encontrados; x_act, x_sig];
    endif

    x_act = x_sig; % se avanza al siguiente punto
    f_act = f_sig;
  endwhile

  % Resumen del barrido.
  if isempty(intervalos_encontrados)
    disp('No se encontraron cambios de signo en el rango analizado.');
  else
    fprintf('\nRaíz separada con éxito para el paso siguiente en el intervalo:\n');
    fprintf('[%g, %g]\n', intervalos_encontrados');
```

```
  endif

  intervalos = intervalos_encontrados;
endfunction
```

Intervalo Medio (bisección)

```
function [raiz, iter] = intervalo_medio(f, a, b, tol, max_iter)
  % INTERVALO_MEDIO - Bisección: parte el intervalo a la mitad y se queda con
  % la mitad donde f cambia de signo.
  % Entradas:
  %   f        - función a la que se le busca la raíz, ej: @(x) x.^2 - 2
  %   a, b     - extremos del intervalo inicial (se pasa un valor por referencia)
  %   tol      - cota de error: se frena cuando (b - a)/2 <= tol
  %   max_iter - tope de iteraciones para que no quede en loop infinito
  % Salidas:
  %   raiz - punto medio del intervalo final, aproximación de la raíz
  %   iter - cantidad de iteraciones realizadas
  % Converge siempre que haya cambio de signo, pero lento: unas log2((b-a)/tol)
  % iteraciones, sin importar la forma de f.

  if (nargin < 4)
    error("intervalo_medio: se requieren al menos 4 argumentos (f, a, b, tol)");
  endif
  if (nargin < 5)
    max_iter = 1000;
  endif
  if (!isa(f, "function_handle"))
    error("intervalo_medio: f debe ser un function_handle, ej: @(x) x.^2 - 2");
  endif
  if (!isnumeric(tol) || !isscalar(tol) || !isfinite(tol) || !(tol > 0))
    error("intervalo_medio: tol debe ser un numero positivo");
  endif

  % Teorema de Bolzano: sin cambio de signo no se puede garantizar la raíz.
  if (f(a) * f(b) > 0)
    error('f(a) y f(b) deben tener signos opuestos.');
  endif

  iter = 0;

  while ((b - a) / 2 > tol && iter < max_iter)
    c = (a + b) / 2;
    f_c = f(c);

    % Caso exacto: se cayó justo en la raíz, no hay nada más que refinar.
    if (f_c == 0)
      a = c;
      b = c;
      iter++;
      break;
    endif

    % Se descarta la mitad donde f no cambia de signo.
    if (f(a) * f_c < 0)
      b = c;
    else
      a = c;
    endif

    iter++;
  endwhile

  % La aproximación es siempre el punto medio del intervalo que quedó.
  raiz = (a + b) / 2;
endfunction
```

Interpolación Lineal

```
function [raiz, iter, tabla] = interpolacion_lineal(f, a, b, tol, max_iter)
  % INTERPOLACION_LINEAL - Método de interpolación lineal.
  % Aproxima la raíz con la intersección de la cuerda entre (a, f(a)) y (b, f(b))
  % con el eje x, en vez de con el punto medio como hace la bisección:
  %   c = b - f(b)*(b - a)/(f(b) - f(a))
  % Entradas:
  %   f        - función a la que se le busca la raíz, ej: @(x) x.^2 - 2
  %   a, b     - extremos del intervalo inicial (se pasa un valor por referencia)
  %   tol      - cota de error: se frena cuando |c_n - c_(n-1)| <= tol
  %   max_iter - tope de iteraciones para que no quede en loop infinito
  % Salidas:
  %   raiz  - última intersección con el eje x calculada
  %   iter  - cantidad de iteraciones realizadas
  %   tabla - matriz Nx4 con [iter, a, b, c] por iteración, para mostrar el avance
  % Suele converger más rápido que la bisección, pero si f es muy curva un extremo
  % se queda pegado; por eso el error se mide como |c_n - c_(n-1)| y no como (b-a)/2.

  if (nargin < 4)
    error("interpolacion_lineal: se requieren al menos 4 argumentos (f, a, b, tol)");
  endif
  if (nargin < 5)
    max_iter = 1000;
  endif
  if (!isa(f, "function_handle"))
    error("interpolacion_lineal: f debe ser un function_handle, ej: @(x) x.^2 - 2");
  endif
  if (!isnumeric(tol) || !isscalar(tol) || !isfinite(tol) || !(tol > 0))
    error("interpolacion_lineal: tol debe ser un numero positivo");
  endif

  % Teorema de Bolzano: sin cambio de signo no se puede garantizar la raíz.
  if (f(a) * f(b) > 0)
    error('f(a) y f(b) deben tener signos opuestos.');
  endif

  iter = 0;
  raiz = a;
  c_anterior = [];
  filas = [];

  while (true)
    f_a = f(a);
    f_b = f(b);

    % c = a - f(a)*(b-a)/(f(b)-f(a)): la misma cuerda, pero sin cancelar
    % cuando f(a) y f(b) son grandes y b - a es chico.
    raiz = a - f_a * (b - a) / (f_b - f_a);
    f_raiz = f(raiz);

    iter++;
    filas(end + 1, :) = [iter, a, b, raiz]; %#ok<AGROW>

    % Caso exacto: se cayó justo en la raíz.
    if (f_raiz == 0)
      break;
    endif

    % Se descarta el tramo donde f no cambia de signo.
    if (f_a * f_raiz < 0)
      b = raiz;
    else
      a = raiz;
    endif
```

```
    % Criterio de parada: dos intersecciones sucesivas difieren menos que la cota.
    if (!isempty(c_anterior) && abs(raiz - c_anterior) <= tol)
      break;
    endif
    c_anterior = raiz;

    if (iter >= max_iter)
      disp('Se alcanzó el tope de iteraciones sin cumplir la cota de error.');
      break;
    endif
  endwhile

  tabla = filas;
endfunction
```

Newton-Raphson

```
function [x_raiz, iter, convergio] = newton_raphson(F, x0, tol)
  % NEWTON_RAPHSON - Metodo de Newton-Raphson: x_{k+1} = x_k - F(x_k)/F'(x_k).
  % Entradas:
  %   F   - funcion a la que se le busca la raiz, ej: @(x) x.^2 - 2
  %   x0  - valor inicial (elegir segun Fourier: F(x0)*F''(x0) > 0)
  %   tol - tolerancia de parada: se frena cuando |x_sig - x_act| < tol
  % Salidas:
  %   x_raiz    - aproximacion de la raiz
  %   iter      - cantidad de iteraciones realizadas
  %   convergio - false si la derivada se anula (no se puede continuar)
  % Ejemplo:
  %   F = @(x) x.^2 - 2;  [x, it, ok] = newton_raphson(F, 1, 1e-4);

  if (nargin != 3)
    error("newton_raphson: se requieren 3 argumentos (F, x0, tol)");
  endif
  if (!isa(F, "function_handle"))
    error("newton_raphson: F debe ser un function_handle, ej: @(x) x.^2 - 2");
  endif
  if (!isnumeric(x0) || !isscalar(x0) || !isfinite(x0))
    error("newton_raphson: x0 debe ser un numero escalar");
  endif
  if (!isnumeric(tol) || !isscalar(tol) || !isfinite(tol) || !(tol > 0))
    error("newton_raphson: tol debe ser un numero positivo");
  endif

  % F'(x) se aproxima por diferencia central, con paso h.
  h = 1e-4;

  dF = @(x) (F(x + h) - F(x - h)) / (2 * h);

  x_act = x0;
  iter = 0;

  % Chequeo inicial: si la derivada se anula en x0 no se puede avanzar.
  if (dF(x_act) == 0)
    disp("Derivada nula en x0, el metodo no puede continuar");
    convergio = false;
    x_raiz = x_act;
    return;
  endif

  % error_k arranca en 1 para entrar al bucle y se actualiza con |x_sig - x_act|.
  error_k = 1;
  while (error_k > tol)
    fx = F(x_act);
    dfx = dF(x_act);

    % Si la derivada se anula a mitad de camino, se sale sin romper todo.
    if (dfx == 0)
      disp("Derivada nula durante la iteracion, el metodo se detiene");
      convergio = false;
      x_raiz = x_act;
      return;
    endif

    x_sig = x_act - (fx / dfx); % paso de Newton-Raphson
    error_k = abs(x_sig - x_act); % medida del avance
    x_act = x_sig; % se avanza al siguiente punto
    iter++;
  endwhile
```

```
  x_raiz = x_act;
  convergio = true;
endfunction
```

Iteración (punto fijo)

```
function [x_raiz, iter, convergio] = iteracion(g, x0, tol)
  % ITERACION - Metodo de punto fijo: resuelve x = g(x) iterando x_{k+1} = g(x_k).
  % Entradas:
  %   g   - funcion de iteracion, ej: @(x) 200 + 40*log(400/(500-x))
  %   x0  - valor inicial
  %   tol - tolerancia de parada: se frena cuando |x_sig - x_act| < tol
  % Salidas:
  %   x_raiz    - aproximacion del punto fijo
  %   iter      - cantidad de iteraciones realizadas
  %   convergio - true si se alcanzo la tolerancia
  % Ejemplo (con ./ para que fplot evalue vectores):
  %   g = @(x) 200 + 40*log(400./(500-x));  [x, it, ok] = iteracion(g, 0, 1e-4);

  if (nargin != 3)
    error("iteracion: se requieren 3 argumentos (g, x0, tol)");
  endif
  if (!isa(g, "function_handle"))
    error("iteracion: g debe ser un function_handle, ej: @(x) x.^2 - 2");
  endif
  if (!isnumeric(x0) || !isscalar(x0) || !isfinite(x0))
    error("iteracion: x0 debe ser un numero escalar");
  endif
  if (!isnumeric(tol) || !isscalar(tol) || !isfinite(tol) || !(tol > 0))
    error("iteracion: tol debe ser un numero positivo");
  endif

  % Para chequear la condicion de convergencia |g'(x0)| < 1 hace falta g'(x0);
  % se aproxima por diferencia central con paso h.
  h = 1e-4;

  dg = @(x) (g(x + h) - g(x - h)) / (2 * h);

  % Si no se cumple la condicion, se avisa y se sale sin iterar.
  if (abs(dg(x0)) >= 1)
    disp("El metodo no converge");
    convergio = false;
    x_raiz = x0;
    iter = 0;
    return;
  endif

  x_act = x0;
  iter = 0;

  while (true)
    x_sig = g(x_act); % nuevo valor a partir del actual
    iter++;
    if (abs(x_sig - x_act) < tol)
      break;
    endif
    x_act = x_sig; % se avanza al siguiente punto
  endwhile

  x_raiz = x_sig;
  convergio = true;
endfunction
```

Aceleración de Aitken

```
function [x_raiz, iter, convergio] = aceleracion(g, x0, tol)
  % ACELERACION - Aceleracion de Aitken delta-cuadrado para x = g(x).
  %   x1 = g(x_act), x2 = g(x1)
  %   x_sig = x_act - (x1 - x_act)^2 / (x2 - 2*x1 + x_act)
  % Entradas:
  %   g   - funcion de iteracion, ej: @(x) 200 + 40*log(400./(500-x))
  %   x0  - valor inicial
  %   tol - tolerancia de parada: se frena cuando |x_sig - x_act| < tol
  % Salidas:
  %   x_raiz    - aproximacion acelerada del punto fijo
  %   iter      - cantidad de iteraciones realizadas
  %   convergio - false si el denominador se anula (no se puede acelerar)
  % Ejemplo (con ./ para que fplot evalue vectores):
  %   g = @(x) 200 + 40*log(400./(500-x));  [x, it, ok] = aceleracion(g, 0, 1e-4);
  % No exige |g'(x0)| < 1: Aitken converge cuadraticamente aun donde el punto
  % fijo simple diverge, siempre que el denominador no sea nulo.

  if (nargin != 3)
    error("aceleracion: se requieren 3 argumentos (g, x0, tol)");
  endif
  if (!isa(g, "function_handle"))
    error("aceleracion: g debe ser un function_handle, ej: @(x) x.^2 - 2");
  endif
  if (!isnumeric(x0) || !isscalar(x0) || !isfinite(x0))
    error("aceleracion: x0 debe ser un numero escalar");
  endif
  if (!isnumeric(tol) || !isscalar(tol) || !isfinite(tol) || !(tol > 0))
    error("aceleracion: tol debe ser un numero positivo");
  endif

  x_act = x0;
  iter = 0;

  % En cada paso se evalua g dos veces y se aplica la formula de Aitken.
  while (true)
    x1 = g(x_act); % primera iterada de punto fijo
    x2 = g(x1);    % segunda iterada de punto fijo
    den = x2 - 2 * x1 + x_act; % denominador de Aitken

    % Si el denominador se anula, la aceleracion no esta definida.
    if (den == 0)
      disp("Denominador nulo en Aitken, el metodo se detiene");
      convergio = false;
      x_raiz = x_act;
      return;
    endif

    x_sig = x_act - ((x1 - x_act)^2) / den; % valor acelerado
    iter++;
    if (abs(x_sig - x_act) < tol)
      break;
    endif
    x_act = x_sig; % se avanza al valor acelerado
  endwhile

  x_raiz = x_sig;
  convergio = true;
endfunction
```